

Core Game Setup (foundation)

Before adding mechanics, define a slightly richer farm:

- Grid-based farm (e.g., 5×5 tiles)
- Each tile has:
 - crop type
 - state: *dry / watered / growing / ready / dead*
- Player (agent) can:
 - move
 - water
 - plant
 - harvest

This gives you enough structure to support everything below.

1. Task Sequencing → “Farming Workflow System”

Mechanic

Each crop requires a **multi-step lifecycle**:

- till soil → plant → water → wait → harvest

Implementation

- Each tile stores a `task_state`
- Some actions only unlock after previous ones

Example test

“Prepare that plot completely”

👉 Agent must generate:

1. Move
2. Till
3. Plant
4. Water

2. Spatial Reasoning → “Zoned Farm Layout”

Mechanic

Divide farm into:

- left / right
- top / bottom
- named zones (Zone A, B)

Implementation

- Tag tiles with coordinates + labels

Example test

"Water the crops on the far left"

👉 Requires:

- Distance calculation
- Region filtering

3. Selective Attention → "Crop Conditions"

Mechanic

Each crop has attributes:

- moisture level
- health
- size

Implementation

- Numeric values (e.g., moisture: 0–100)

Example test

"Only water the really dry ones"

👉 Define threshold (e.g., <30%)

4. Priority & Optimization → "Resource Constraints"

Mechanic

- Limited water tank
- Time pressure (day/night cycle)

Implementation

- Water decreases per action

- Crops decay over time

Example test

“Use water carefully”

👉 Agent must:

- Skip healthy crops
- Prioritize worst ones

5. Interruptions → “Cancelable Actions”

Mechanic

Actions take time (e.g., 2–3 seconds)

Implementation

- Action queue
- Interrupt flag

Example test

“Stop! Go back!”

👉 Agent cancels current task and replans

6. Ambiguity → “Multiple Similar Targets”

Mechanic

- Several identical crops in view

Implementation

- No automatic disambiguation

Example test

“Water that one”

👉 System should:

- Pause
- Ask: “Which one?”

7. Memory → “Task History System”

Mechanic

Store:

- last action
- last target
- last region

Implementation

- Simple memory object

Example test

“Do the same for the other side”

👉 Requires:

- Recall previous action
- Apply to new region

8. Delegation → “Multiple Workers (Optional but powerful)”

Mechanic

Add:

- 2nd worker or drone

Implementation

- Assign tasks to agents

Example test

“You take the left, handle the right”

👉 Task splitting logic

9. Noisy Input → “Speech Variability Mode”

Mechanic

Feed system:

- incomplete transcripts
- filler words

Example test

"uh yeah the plants... water them I guess"

👉 Must still resolve intent

10. Safety Constraints → "Rule System"

Mechanic

Rules like:

- don't overwater (>90%)
- don't waste water

Implementation

- Hard constraints in planner

Example test

"Water everything repeatedly"

👉 Agent should refuse or adjust

11. Explainability → "Action Narration"

Mechanic

Agent outputs reasoning text

Example

"Watering this crop because moisture is low (20%)"

Implementation

- LLM generates explanation alongside action

12. Temporal Reasoning → “Time-Based Events”

Mechanic

- Crops dry over time
- Growth stages

Implementation

- Tick system (every few seconds)

Example test

“Check again later”

👉 Agent schedules future action

13. Goal-Oriented Mode → “Autonomous Farming Mode”

Mechanic

User gives high-level goal:

- “Keep the farm healthy”

Implementation

- Continuous loop:
 - observe → plan → act

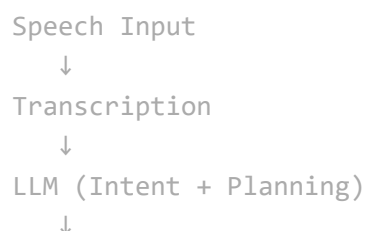
Example

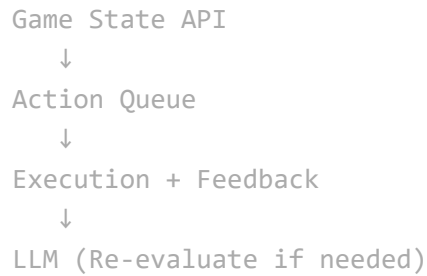
Agent independently:

- waters dry crops
- harvests ready ones

How to Tie It Together (Architecture)

Your system should look like:





Turn This Into a Strong Research Project

Each mechanic becomes a **test dimension**:

Capability	Metric
Planning	Steps completed correctly
Ambiguity	# clarification turns
Memory	Context accuracy
Optimization	Resource efficiency
Robustness	Success under noisy input



Key Insight

Don't think:

"What else can the player do?"

Think:

"What kinds of *thinking* should the agent demonstrate?"

Your farm is just a **controlled sandbox for intelligence testing**.

If you want next step, I can:

- ↳ design a minimal Unity / Python implementation plan
- ↳ create a set of 50+ test voice commands mapped to these mechanics

